

Proyecto 2 — Proxy/Cache de Aplicación con Geolocalización

Documentación del proyecto

Tecnológico de Costa Rica

Escuela de Ingeniería en Computación

Curso: Redes IC7602

Proyecto: Proyecto 2 - Proxy/Cache de Aplicación con Geolocalización

Semestre: I Semestre 2026

Integrantes

- Mariela Solano Gómez
 - Alejandra Delgado Pérez
 - Joshua Obando Castro
 - Roilin Navarro Vargas
 - Esteban Cortes
-

Tabla de contenidos

1. [Introducción](#)
 2. [Arquitectura general del proyecto](#)
 3. [Estructura del repositorio](#)
 4. [DNS Interceptor](#)
 5. [Zonal Cache](#)
 6. [REST API Node.js](#)
 7. [UI React + Vite + Tailwind CSS](#)
 8. [REST API Java](#)
 9. [Apache Server](#)
 10. [Firebase Firestore](#)
 11. [Ejecución del proyecto](#)
 12. [Despliegue con Docker y Helm](#)
 13. [Pruebas realizadas](#)
 14. [Recomendaciones](#)
 15. [Conclusiones](#)
-

Introducción

Este proyecto consiste en implementar un proxy/cache de aplicación con geolocalización. La idea principal es que el sistema se coloque entre los usuarios y los servidores origen, de manera que las solicitudes puedan ser interceptadas, autenticadas y almacenadas en caché.

El proyecto combina varios temas de redes, principalmente capa de aplicación y capa de transporte. Se trabaja con servicios HTTP, DNS, UDP, TCP, caché en disco, autenticación, Firebase, Kubernetes, Docker y Helm Charts.

El flujo general del sistema inicia cuando un usuario intenta acceder a un dominio. El DNS Interceptor recibe la consulta DNS y redirige la solicitud hacia una Zonal Cache. La selección de la caché puede depender de la ubicación del usuario, usando información de IP to Country. Luego, la Zonal Cache revisa si el recurso solicitado ya se encuentra guardado en disco. Si está en caché, responde directamente. Si no está, consulta el servidor origen, guarda el recurso y lo devuelve al usuario.

Además, el proyecto incluye una interfaz web administrativa. Desde la UI se pueden registrar dominios, validar propiedad por medio de registros TXT, configurar URLs, definir políticas de caché, crear usuarios, generar API Keys y modificar la configuración que luego usa la Zonal Cache.

También se implementaron dos servidores origen para pruebas: una REST API en Java con Spring Boot y un servidor Apache con una página HTML básica. Estos componentes sirven para demostrar que la caché puede trabajar tanto con respuestas JSON como con archivos estáticos.

Arquitectura general del proyecto

El sistema tiene dos grandes flujos: el flujo de usuario y el flujo administrativo.

Flujo de usuario

```
Usuario
  |
  | Consulta DNS / solicitud HTTP
  v
DNS Interceptor
  |
  | Redirección según dominio / región
  v
Zonal Cache
  |
  | HIT o MISS
  v
Servidor origen
  |
  | REST API Java / Apache / sitio externo
  v
Respuesta al usuario
```

Flujo administrativo

```
Administrador
  |
  | Login / configuración
```

```

  v
UI React
  |
  | Peticiones HTTP
  v
REST API Node.js
  |
  | Lectura / escritura
  v
Firebase Firestore
```

Flujo de configuración dinámica

```

Zonal Cache
  |
  | Consulta configuración del dominio
  v
REST API Node.js
  |
  | Lee datos
  v
Firebase Firestore
```

En resumen, el sistema funciona así:

1. El administrador configura dominios, URLs, TTL, tamaño de caché, políticas de reemplazo y autenticación desde la UI.
2. La UI se comunica con la REST API Node.js.
3. La REST API Node.js guarda la configuración en Firebase Firestore.
4. La Zonal Cache consulta la configuración desde Firebase mediante la API.
5. El usuario hace una solicitud.
6. El DNS Interceptor decide hacia cuál caché zonal debe ir la solicitud.
7. La Zonal Cache valida si el recurso requiere autenticación.
8. Si la autenticación es correcta, revisa si el recurso está en caché.
9. Si hay HIT, responde desde disco.
10. Si hay MISS, consulta el origen, guarda el recurso y responde.

Estructura del repositorio

La estructura general del proyecto es la siguiente:

```

P2/
├─ api/
│   └─ routes/
│   └─ middleware/
```

- ├─ firebase.js
- ├─ server.js
- ├─ package.json
- ├─ vercel.json
- └─ .env.example
- ─ ui/
 - ├─ src/
 - ├─ api/
 - ├─ components/
 - ├─ mocks/
 - ├─ pages/
 - ├─ store/
 - ├─ App.jsx
 - └─ main.jsx
 - ├─ package.json
 - ├─ vite.config.js
 - ├─ vercel.json
 - └─ .env.example
- ─ dns-interceptor/
 - ├─ Cargo.toml
 - ├─ Dockerfile
 - ├─ README.md
 - └─ src/
 - ├─ main.rs
 - ├─ dns_parser.rs
 - ├─ dns_response.rs
 - ├─ geo_locator.rs
 - └─ query_handler.rs
- ─ zonal-cache/
 - ├─ Cargo.toml
 - ├─ Dockerfile
 - ├─ config.json
 - └─ src/
 - ├─ main.rs
 - ├─ auth/
 - ├─ cache/
 - └─ firebase/
- ─ java-rest-api/
 - ├─ Dockerfile
 - ├─ pom.xml
 - ├─ README.md
 - └─ src/
 - ├─ main/
 - ├─ java/redes/api/
 - └─ resources/
 - └─ test/
 - └─ java/redes/api/
- ─ apache-server/
 - ├─ Dockerfile

```
├── index.html
├── style.css
└── README.md

├── charts/
│   ├── dns-interceptor/
│   ├── zonal-cache/
│   ├── java-rest-api/
│   ├── apache-server/
│   └── ui/
├── build.sh
├── install.sh
└── README.md
```

DNS Interceptor

Descripción

El DNS Interceptor es un servidor DNS ligero implementado en Rust. Su función principal es recibir consultas DNS por UDP, analizar el paquete recibido, extraer el dominio consultado y responder dinámicamente con una dirección IP de caché regional basada en la ubicación geográfica del cliente.

Dentro del Proyecto 2, este componente funciona como punto de entrada para los usuarios. Cuando un cliente intenta acceder a un dominio, el DNS Interceptor procesa la consulta DNS, obtiene la IP de origen del cliente y utiliza una función RPC en Supabase para determinar el país desde donde proviene la solicitud.

Con esa información, el sistema selecciona automáticamente una caché zonal según la región del usuario. De esta manera, el cliente recibe una respuesta DNS tipo A con la IP de la caché regional correspondiente, sin tener que conocer internamente cómo se hizo la selección.

Este componente fue implementado en Rust y se prepara para ejecutarse en Kubernetes mediante Helm Charts.

Responsabilidades principales

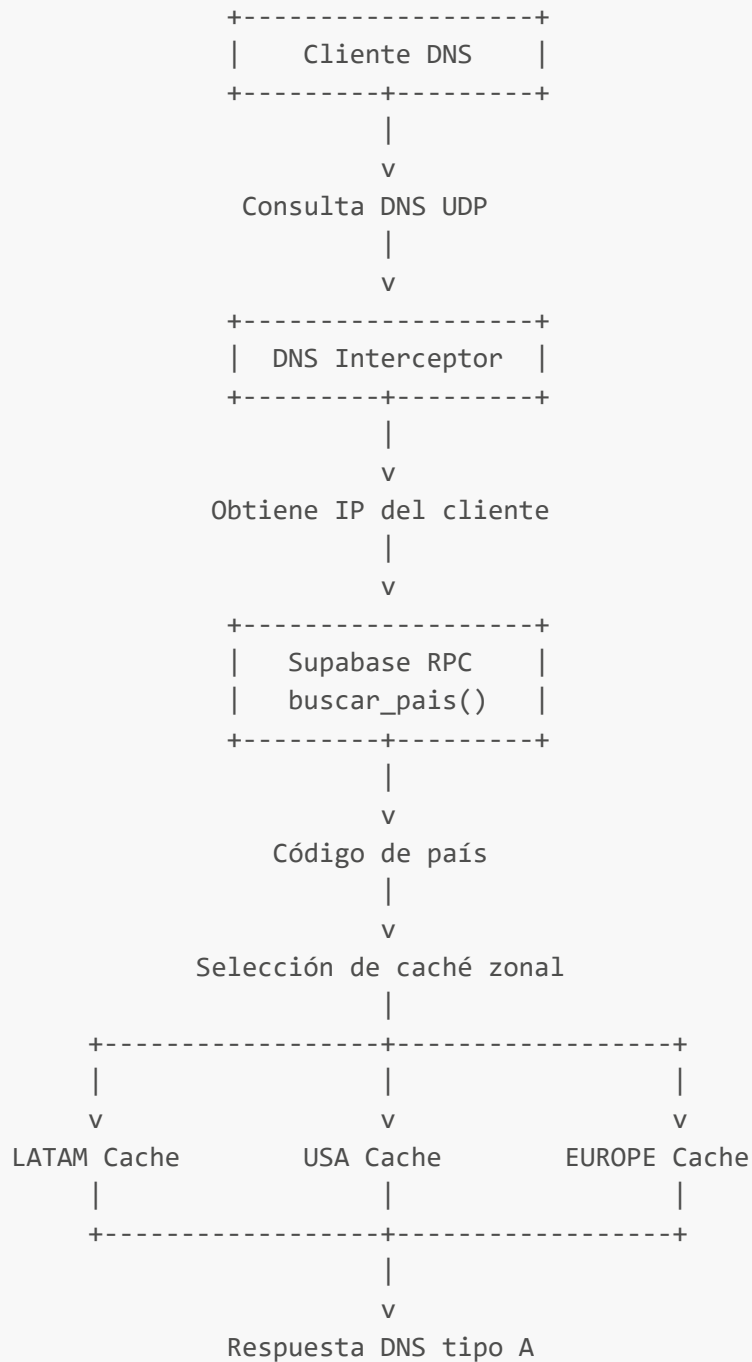
El DNS Interceptor se encarga de:

- Escuchar consultas DNS usando UDP.
- Recibir paquetes DNS desde clientes.
- Parsear manualmente el encabezado DNS.
- Extraer el dominio consultado desde el QNAME.
- Validar si el paquete corresponde a una consulta DNS estándar.
- Obtener la IP del cliente que hizo la consulta.
- Consultar Supabase mediante una función RPC.
- Determinar el país de origen de la IP cliente.
- Seleccionar automáticamente una caché regional.

- Construir respuestas DNS tipo A con la IP de la caché seleccionada.
- Enviar la respuesta DNS final al cliente.
- Ejecutarse dentro de contenedores Docker.
- Desplegarse en Kubernetes mediante Helm Charts.

Arquitectura del DNS Interceptor

El flujo general del DNS Interceptor es el siguiente:



Relación con la Zonal Cache

El DNS Interceptor no almacena contenido HTTP en caché. Su trabajo es responder la consulta DNS con la IP de la caché regional que corresponde según la ubicación del cliente.

La Zonal Cache es la que se encarga de:

- recibir la solicitud HTTP;
- consultar configuración desde Firebase;
- validar autenticación;
- revisar si el recurso está en caché;
- consultar el servidor origen si hay MISS;
- guardar recursos en disco;
- servir recursos cacheados.

La relación entre ambos componentes es:

```
graph TD
    DI[DNS Interceptor] -- "Responde con IP de caché regional" --> ZC[Zonal Cache]
    ZC -- "Procesa HTTP, autenticación y caché" --> SO[Servidor origen]
```

En Kubernetes, el DNS Interceptor puede responder con la IP asociada a una caché zonal o servicio configurado para una región específica.

Archivos principales

```
dns-interceptor/
├── Cargo.toml
├── Dockerfile
├── README.md
├── src/
│   ├── main.rs
│   ├── dns_parser.rs
│   ├── dns_response.rs
│   ├── geo_locator.rs
│   └── query_handler.rs
```

Archivo	Descripción
<code>main.rs</code>	Punto de entrada del programa. Abre el socket UDP, escucha consultas y crea un hilo por solicitud.
<code>dns_parser.rs</code>	Se encarga de parsear el encabezado DNS y extraer el dominio consultado.

Archivo	Descripción
<code>dns_response.rs</code>	Construye respuestas DNS tipo A usando la IP regional seleccionada y el TTL configurado.
<code>geo_locator.rs</code>	Cliente encargado de consultar Supabase RPC para determinar el país del cliente.
<code>query_handler.rs</code>	Contiene la lógica principal para validar la consulta, obtener la IP del cliente, consultar el país y seleccionar la caché regional.

Funcionamiento interno

1. Recepción de consulta DNS

El Interceptor abre un socket UDP en el puerto configurado.

Por defecto usa el puerto `53`, que es el puerto estándar de DNS.

```
UdpSocket::bind("0.0.0.0:53")
```

Cada consulta es procesada en un hilo independiente mediante `thread::spawn`, lo cual permite atender varias solicitudes sin bloquear todo el servidor.

El puerto puede configurarse con la variable de entorno:

```
DNS_PORT=53
```

Para pruebas locales también puede usarse otro puerto, por ejemplo:

```
DNS_PORT=5353
```

2. Parseo del paquete DNS

Cuando llega un paquete DNS, el Interceptor analiza el encabezado.

Se extraen datos como:

- Transaction ID.
- QR Flag.
- Opcode.
- Dominio consultado.

El Transaction ID es importante porque la respuesta debe usar el mismo ID de la consulta original. Así el cliente puede asociar correctamente la respuesta con la consulta que envió.

3. Validación de consulta estándar

El Interceptor revisa si el paquete recibido corresponde a una query estándar.

Una consulta estándar debe cumplir:

```
QR = 0  
OPCODE = 0
```

Esto significa que el paquete recibido es una consulta DNS normal. Actualmente el Interceptor se enfoca en este tipo de consultas para construir respuestas DNS tipo A.

4. Extracción del dominio

El dominio se obtiene desde el campo QNAME del paquete DNS.

Por ejemplo, el dominio:

```
www.example.com
```

en el paquete DNS viene representado por labels:

```
03 www 07 example 03 com 00
```

El Interceptor reconstruye esos labels y obtiene el dominio final:

```
www.example.com
```

Actualmente el dominio se utiliza principalmente para logging, trazabilidad y para mantener claridad sobre qué recurso está consultando el cliente.

5. Obtención de la IP del cliente

El Interceptor obtiene la IP del cliente a partir de la dirección origen del paquete UDP.

En el código, esta IP se obtiene de la dirección `src` recibida por el socket.

Ejemplo conceptual:

```
let client_ip = src.ip().to_string();
```

Esa IP es la que se utiliza para consultar Supabase y determinar el país de origen del cliente.

Geolocalización del cliente

Para determinar la ubicación del cliente, el DNS Interceptor ejecuta una función RPC en Supabase llamada `buscar_pais`.

Endpoint usado:

```
POST /rest/v1/rpc/buscar_pais
```

Payload enviado:

```
{
  "client_ip": "181.193.10.50"
}
```

Respuesta esperada:

```
[
  {
    "country_code": "CR"
  }
]
```

El campo más importante es `country_code`, porque con ese valor el Interceptor decide qué caché regional debe responder.

Algoritmo de selección de Zonal Cache

En la implementación actual, el DNS Interceptor sí participa directamente en la selección de la caché regional.

El algoritmo usado es el siguiente:

1. Recibir una consulta DNS por UDP.
2. Parsear el header DNS.
3. Verificar que sea una consulta estándar:
QR = 0
Opcode = 0
4. Extraer el dominio consultado desde QNAME.

5. Obtener la IP del cliente desde la dirección origen del paquete UDP.
6. Consultar Supabase RPC:
 POST /rest/v1/rpc/buscar_pais
7. Enviar la IP del cliente:

```
{
  "client_ip": "181.193.10.50"
}
```
8. Recibir el código de país:

```
[
  {
    "country_code": "CR"
  }
]
```
9. Seleccionar la caché regional según el país:
 CR, NI, PA, GT, HN -> ZONAL_CACHE_LATAM_IP
 US, CA, MX -> ZONAL_CACHE_USA_IP
 Otros países -> ZONAL_CACHE_EUROPE_IP
10. Construir una respuesta DNS tipo A con la IP seleccionada.
11. Enviar la respuesta DNS al cliente.

En forma de flujo:

```

Cliente DNS
↓
DNS Interceptor
↓
Extrae dominio consultado
↓
Obtiene IP del cliente
↓
Supabase RPC buscar_pais(client_ip)
↓
Obtiene código de país
↓
Selecciona caché regional
↓
Construye respuesta DNS tipo A
↓
Cliente recibe IP de la Zonal Cache

```

Ejemplo para un cliente de Costa Rica:

```
client_ip = 181.193.10.50
↓
Supabase RPC devuelve CR
↓
CR pertenece a LATAM
↓
Se selecciona ZONAL_CACHE_LATAM_IP
↓
Respuesta DNS:
app.miempresa.com -> 10.10.1.10
```

Ejemplo para un cliente de Estados Unidos:

```
client_ip = IP de Estados Unidos
↓
Supabase RPC devuelve US
↓
US pertenece a Norteamérica
↓
Se selecciona ZONAL_CACHE_USA_IP
```

Ejemplo para otros países:

```
País no reconocido en LATAM o Norteamérica
↓
Se selecciona ZONAL_CACHE_EUROPE_IP
```

Regiones configuradas

LATAM

Países configurados para la región LATAM:

```
CR
NI
PA
GT
HN
```

Variable utilizada:

```
ZONAL_CACHE_LATAM_IP
```

Norteamérica

Países configurados para Norteamérica:

```
US  
CA  
MX
```

Variable utilizada:

```
ZONAL_CACHE_USA_IP
```

Europa y otras regiones

Si el país no pertenece a LATAM ni Norteamérica, se utiliza la caché configurada para Europa u otras regiones.

Variable utilizada:

```
ZONAL_CACHE_EUROPE_IP
```

Construcción de respuesta DNS tipo A

Cuando ya se seleccionó la IP de la caché regional, el Interceptor construye manualmente una respuesta DNS tipo A.

La respuesta incluye:

- Transaction ID original.
- Pregunta DNS original.
- Registro tipo A.
- TTL configurable.
- Dirección IPv4 de la caché regional.
- Compresión DNS para referenciar el nombre consultado.

Ejemplo:

```
www.example.com -> 10.10.1.10
```

La respuesta utiliza compresión DNS con:

```
C0 0C
```

Esto permite referenciar el nombre consultado en la sección de respuesta sin repetirlo completo.

Envío de respuesta al cliente

Una vez construida la respuesta DNS, el Interceptor la envía directamente al cliente mediante UDP.

Ejemplo conceptual:

```
socket.send_to(&dns_response, src)?;
```

Con esto, el cliente recibe una respuesta DNS normal tipo A, pero la IP devuelta corresponde a la caché regional seleccionada por el sistema.

Variables de entorno usadas

El DNS Interceptor utiliza variables de entorno para evitar valores quemados en el código y permitir que la configuración cambie según el ambiente.

Variable	Descripción	Ejemplo
DNS_PORT	Puerto UDP donde escucha el Interceptor.	53 o 5353
SUPABASE_URL	URL base del proyecto en Supabase.	https://mi-proyecto.supabase.co
SUPABASE_KEY	API Key usada para consultar la función RPC.	xxxxxxxxxxxxxxxxxx
ZONAL_CACHE_LATAM_IP	IP de la caché regional para Latinoamérica.	10.10.1.10
ZONAL_CACHE_USA_IP	IP de la caché regional para Norteamérica.	10.20.1.10
ZONAL_CACHE_EUROPE_IP	IP de la caché regional para Europa u otras regiones.	10.30.1.10

Ejemplo de configuración local:

```
export DNS_PORT=5353
export SUPABASE_URL=https://mi-proyecto.supabase.co
```

```
export SUPABASE_KEY=xxxxxxxxxxxxxxxxxxx
export ZONAL_CACHE_LATAM_IP=10.10.1.10
export ZONAL_CACHE_USA_IP=10.20.1.10
export ZONAL_CACHE_EUROPE_IP=10.30.1.10
```

En Kubernetes, estas variables se inyectan desde el Helm Chart del DNS Interceptor.

Helm Chart

El componente tiene un Helm Chart en:

```
charts/dns-interceptor/
```

Estructura:

```
charts/dns-interceptor/
├─ Chart.yaml
├─ values.yaml
├─ templates/
│   └─ deployment.yaml
│   └─ service.yaml
```

En el `values.yaml`, el servicio se puede configurar como `NodePort` para exponer UDP:

```
service:
  type: NodePort
  port: 53
  targetPort: 53
  nodePort: 30053
```

Las variables de entorno deben incluir la configuración de Supabase y las IPs regionales:

```
env:
  dnsPort: "53"
  supabaseUrl: "https://mi-proyecto.supabase.co"
  supabaseKey: "xxxxxxxxxxxxxxxxxxx"
  zonalCacheLatamIp: "10.10.1.10"
  zonalCacheUsaIp: "10.20.1.10"
  zonalCacheEuropeIp: "10.30.1.10"
```

En el Deployment se inyectan como variables de entorno:

```
env:  
- name: DNS_PORT  
  value: "53"  
- name: SUPABASE_URL  
  value: "https://mi-proyecto.supabase.co"  
- name: SUPABASE_KEY  
  value: "xxxxxxxxxxxxxxxxxx"  
- name: ZONAL_CACHE_LATAM_IP  
  value: "10.10.1.10"  
- name: ZONAL_CACHE_USA_IP  
  value: "10.20.1.10"  
- name: ZONAL_CACHE_EUROPE_IP  
  value: "10.30.1.10"
```

Por seguridad, la API Key de Supabase no debería quedar expuesta en repositorios públicos. Lo recomendable es manejarla mediante variables de entorno o Kubernetes Secrets.

Ejecución local esperada

```
cd dns-interceptor  
cargo run
```

También se puede ejecutar en modo release:

```
cd dns-interceptor  
cargo run --release
```

Compilación local

```
cd dns-interceptor  
cargo build --release
```

Construcción con Docker

Desde la raíz del proyecto:

```
docker build -t dns-interceptor:latest ./dns-interceptor
```

Instalación con Helm

Desde la raíz del proyecto:

```
helm upgrade --install dns-interceptor ./charts/dns-interceptor
```

Verificar:

```
kubectl get pods  
kubectl get svc
```

Prueba sugerida con nslookup

Si el servicio usa el puerto estándar:

```
nslookup <dominio> 127.0.0.1
```

Si se expone por un puerto diferente:

```
nslookup -port=<puerto> <dominio> 127.0.0.1
```

Como en el Helm Chart se puede usar **NodePort 30053**, la prueba esperada desde la máquina local sería:

```
nslookup -port=30053 app.miempresa.com 127.0.0.1
```

También se puede probar con **dig**:

```
dig @127.0.0.1 -p 30053 app.miempresa.com
```

Dominios de prueba

Para probar el DNS Interceptor se pueden usar dominios de prueba como:

```
app.miempresa.com  
cache-test.com  
example.com
```

El dominio consultado se usa principalmente para trazabilidad y logging. La IP final depende de la geolocalización del cliente y de las variables de entorno configuradas para cada región.

Logs esperados

Durante la ejecución del DNS Interceptor se esperan logs similares a los siguientes:

```
[DNS Interceptor] Escuchando en UDP/0.0.0.0:53
[DNS_HANDLER] Consulta recibida desde 181.193.10.50
[DNS_HANDLER] Dominio consultado: app.miempresa.com
[DNS_HANDLER] Consultando Supabase RPC buscar_pais
[DNS_HANDLER] País detectado: CR
[DNS_HANDLER] Región seleccionada: LATAM
[DNS_HANDLER] IP de caché seleccionada: 10.10.1.10
[DNS_RESPONSE] Respuesta DNS tipo A construida correctamente
[DNS_HANDLER] Respuesta enviada al cliente
```

Para un cliente de Norteamérica:

```
[DNS_HANDLER] Consulta recibida desde <ip_cliente>
[DNS_HANDLER] País detectado: US
[DNS_HANDLER] Región seleccionada: USA
[DNS_HANDLER] IP de caché seleccionada: 10.20.1.10
```

Para un país no incluido en LATAM o USA:

```
[DNS_HANDLER] Consulta recibida desde <ip_cliente>
[DNS_HANDLER] País detectado: ES
[DNS_HANDLER] Región seleccionada: EUROPE
[DNS_HANDLER] IP de caché seleccionada: 10.30.1.10
```

Evidencia de pruebas con nslookup

Prueba para cliente ubicado en Latinoamérica:

```
nslookup -port=30053 app.miempresa.com 127.0.0.1
```

Resultado esperado:

```
Server: 127.0.0.1
Address: 127.0.0.1#30053
```

```
Name:      app.miempresa.com
Address: 10.10.1.10
```

Conclusión:

El DNS Interceptor recibió la consulta, obtuvo la IP del cliente, consultó Supabase RPC, detectó la región correspondiente y respondió con la IP de la caché LATAM.

Prueba con **dig**:

```
dig @127.0.0.1 -p 30053 app.miempresa.com
```

Resultado esperado:

```
;; ANSWER SECTION:
app.miempresa.com. 300 IN A 10.10.1.10
```

Conclusión:

La respuesta DNS tipo A fue generada correctamente usando la IP regional seleccionada.

Pruebas unitarias

El módulo incluye pruebas para validar partes importantes del DNS Interceptor.

Ejecutar:

```
cd dns-interceptor
cargo test
```

Pruebas principales:

- parseo correcto del encabezado DNS;
- detección de paquetes demasiado cortos;
- extracción correcta de dominios;
- detección de QNAME malformado;
- construcción de respuesta DNS tipo A;

- manejo de IP inválida.

Resultado esperado:

```
test result: ok
```

Explicación de integración con la Zonal Cache en Kubernetes

Dentro de Kubernetes, los componentes se comunican usando los nombres de los Services y las IPs configuradas para cada región.

El DNS Interceptor no procesa HTTP ni almacena contenido. Su trabajo es responder la consulta DNS con la IP de una caché regional.

El flujo integrado sería:

```
Cliente DNS
  ↓
DNS Interceptor
  ↓
Obtiene IP del cliente
  ↓
Supabase RPC buscar_pais(client_ip)
  ↓
Selecciona región
  ↓
Responde con IP de caché regional
  ↓
Cliente accede al recurso HTTP
  ↓
Zonal Cache procesa autenticación y caché
  ↓
Servidor origen
```

La integración esperada es:

```
dns-interceptor
  ↓
zonal-cache LATAM / USA / EUROPE
  ↓
java-rest-api-service / apache-server-service / sitios externos
```

Con esto, el DNS Interceptor se encarga de la entrada DNS y la selección regional, mientras que la Zonal Cache se encarga del procesamiento HTTP, autenticación, almacenamiento en disco y consulta al servidor origen.

Zonal Cache

Descripción

La Zonal Cache es el componente central del proyecto. Su responsabilidad es recibir solicitudes HTTP, revisar si el recurso solicitado ya está almacenado en disco y responder desde caché cuando sea posible.

Si el recurso no existe en caché, la Zonal Cache consulta el servidor origen, guarda el recurso en disco y luego lo devuelve al usuario. Además, aplica mecanismos de autenticación y lee su configuración dinámicamente desde Firebase mediante la REST API Node.js.

La Zonal Cache fue implementada en Rust usando Axum y Tokio.

Responsabilidades principales

La Zonal Cache se encarga de:

- Recibir solicitudes HTTP.
- Consultar configuración dinámica desde Firebase.
- Aplicar autenticación según el dominio o URL.
- Verificar API Keys.
- Validar sesiones de usuario.
- Revisar si un recurso existe en caché.
- Servir recursos desde disco cuando hay HIT.
- Consultar el origen cuando hay MISS.
- Guardar recursos en disco.
- Aplicar algoritmos de reemplazo.
- Soportar recursos HTTP y HTTPS.
- Integrarse con DNS Interceptor, REST API Node.js, REST API Java y Apache Server.

Motor de caché

El motor de caché trabaja con una carpeta en disco donde se guardan los recursos descargados.

Flujo principal:

```
Cliente
  |
  | GET /cache?url={url}
  v
Zonal Cache
  |
  | ¿Existe en HashMap?
  |
  |— Sí: leer archivo de disco y servir respuesta
  |
  |— No: hacer request al origen, guardar en disco y servir respuesta
```

Cuando el caché se llena:

```

Zonal Cache
|
| Detecta límite de tamaño
v
Aplica política de reemplazo
|
| Elimina una entrada
v
Guarda nuevo recurso
```

Rutas principales

Método	Ruta	Descripción
GET	<code>/cache?url={url}</code>	Busca un recurso en caché o lo obtiene del origen.
GET	<code>/api/exists?url={url}</code>	Devuelve el recurso si existe en caché o 404 si no existe.
POST	<code>/api/dns_resolver</code>	Recibe datos en Base64 e intenta reenviarlos al DNS Interceptor.

Caché en disco

Los recursos se guardan en:

```
cache_disco/
```

El nombre del archivo se genera a partir de la URL, eliminando caracteres especiales para que pueda ser almacenado de forma segura en disco.

En memoria se mantiene un `HashMap` con metadata de cada recurso:

```
URL -> metadata
```

La metadata incluye información como:

- ruta del archivo en disco;
- tamaño del recurso;
- fecha de creación;
- último acceso;
- frecuencia de uso;
- política de reemplazo aplicada.

Algoritmos de reemplazo

La Zonal Cache implementa cinco algoritmos de reemplazo:

Algoritmo	Descripción
FIFO	Elimina el recurso que entró primero al caché.
LRU	Elimina el recurso usado menos recientemente.
MRU	Elimina el recurso usado más recientemente.
LFU	Elimina el recurso con menor frecuencia de acceso.
Random	Elimina un recurso aleatorio.

Configuración dinámica

Al iniciar, la Zonal Cache consulta la configuración desde la REST API Node.js, la cual lee los datos desde Firebase.

Flujo:

```
graph TD
    ZonalCache[Zonal Cache] --> GET[GET {CONFIG_API_URL}/domains/{DOMAIN}/config]
    GET --> RESTAPI[REST API Node.js]
    RESTAPI --> Firestore[Consulta Firestore]
    Firestore --> Firebase[Firebase]
    Firebase --> Devuelve[Devuelve configuración]
    Devuelve --> ZonalCacheTTL[Zonal Cache usa TTL, tamaño y política]
```

Si la API no está disponible, el servidor puede arrancar con valores por defecto para evitar que el servicio se caiga.

Variables de entorno

Variable	Valor por defecto	Descripción
DOMAIN	localhost	Dominio a consultar en Firebase.
CONFIG_API_URL	http://localhost:4000	URL base de la API de configuración.
DNS_INTERCEPTOR_URL	http://localhost:5000	URL del DNS Interceptor.

Autenticación en Zonal Cache

La Zonal Cache soporta tres modos principales de autenticación.

1. Sin autenticación

```
auth_type=none
```

El usuario accede directamente al recurso.

2. API Key

```
auth_type=api_key
```

La solicitud debe incluir el encabezado:

```
x-api-key: <clave>
```

Ejemplo:

```
Invoke-WebRequest `
  -Uri http://localhost:8080 `
  -Headers @{ "x-api-key"="abc123" } `
  -UseBasicParsing
```

3. Usuario y contraseña con sesión

```
auth_type=session
```

Cuando el recurso requiere sesión, la Zonal Cache redirige al formulario de autenticación de la UI.

Flujo:

```
Usuario
|
| Accede a recurso protegido
v
Zonal Cache
|
| Detecta auth_type=session
v
Redirección a UI /auth
|
| Usuario ingresa credenciales
v
```



```
REST API Node.js valida usuario
|
| Devuelve token
v
Zonal Cache valida token
|
| Acceso permitido
v
Motor de caché
```

Pruebas unitarias de Zonal Cache

Se ejecutaron pruebas unitarias para validar los algoritmos de reemplazo:

```
cd zonal-cache
cargo test
```

Pruebas reportadas:

```
fifo_borra_el_primerero_en_llegar
fifo_ignora_accesos_recientes
lru_borra_el_menos_reciente
mru_borra_el_mas_reciente
lfu_borra_el_menos_frecuente
random_borra_alguna_entrada
```

Pruebas manuales reportadas

- HIT/MISS básico con <http://example.com>.
- Eviction con LRU usando límite pequeño.
- Diferencia entre FIFO y LRU.
- Integración con API de configuración.
- HTTPS usando Rustls.
- Validación de `GET /api/exists`.
- Validación de `POST /api/dns_resolver`.
- Ejecución dentro de contenedor Docker.

REST API Node.js

Descripción

La REST API Node.js funciona como backend principal para la UI y como punto de consulta para la Zonal Cache. Este componente se comunica con Firebase Firestore y expone endpoints para administrar usuarios, dominios, URLs, API Keys y configuraciones de caché.

También implementa seguridad usando JWT y bcrypt.

Responsabilidades principales

La REST API Node.js se encarga de:

- Registrar usuarios administradores.
- Iniciar sesión administrativa.
- Generar tokens JWT.
- Validar tokens en endpoints protegidos.
- Guardar contraseñas con hash usando bcrypt.
- Crear, listar y eliminar dominios.
- Verificar dominios mediante registros TXT.
- Crear, modificar y eliminar URLs.
- Crear y eliminar API Keys.
- Crear, modificar y eliminar usuarios por URL.
- Exponer configuración para la Zonal Cache.
- Conectarse con Firebase Firestore.

Tecnologías usadas

Tecnología	Propósito
Node.js	Runtime del backend.
Express	Framework HTTP.
Firebase Admin SDK	Conexión con Firestore.
jsonwebtoken	Creación y validación de JWT.
bcryptjs	Hash seguro de contraseñas.
dotenv	Manejo de variables de entorno.
Vercel	Despliegue serverless.

Endpoints principales

Autenticación

Método	Ruta	Descripción
POST	/auth/register	Registrar administrador.
POST	/auth/login	Iniciar sesión administrativa.
POST	/auth/logout	Cerrar sesión.
POST	/auth/verify	Verificar usuario para acceso desde Zonal Cache.

Dominios

Método	Ruta	Descripción
GET	<code>/domains</code>	Listar dominios.
POST	<code>/domains</code>	Crear dominio y generar registro TXT.
GET	<code>/domains/:domain/config</code>	Obtener configuración para la Zonal Cache.
DELETE	<code>/domains/:domain</code>	Eliminar dominio.
POST	<code>/domains/:domain/verify</code>	Verificar propiedad del dominio con TXT.

URLs

Método	Ruta	Descripción
GET	<code>/domains/:domain/urls</code>	Listar URLs de un dominio.
POST	<code>/domains/:domain/urls</code>	Crear URL.
PUT	<code>/domains/:domain/urls/:urlId</code>	Actualizar URL.
DELETE	<code>/domains/:domain/urls/:urlId</code>	Eliminar URL.

API Keys

Método	Ruta	Descripción
GET	<code>/urls/:urlId/apikeys</code>	Listar API Keys.
POST	<code>/urls/:urlId/apikeys</code>	Generar API Key.
DELETE	<code>/urls/:urlId/apikeys/:keyId</code>	Eliminar API Key.

Usuarios por URL

Método	Ruta	Descripción
GET	<code>/urls/:urlId/users</code>	Listar usuarios.
POST	<code>/urls/:urlId/users</code>	Crear usuario.
PUT	<code>/urls/:urlId/users/:userId</code>	Actualizar usuario.
DELETE	<code>/urls/:urlId/users/:userId</code>	Eliminar usuario.

Seguridad con JWT y bcrypt

El sistema usa JWT para manejar sesiones de forma stateless.

Flujo:

```
Usuario hace login
|
```

```
| email + password
v
REST API Node.js
|
| bcrypt.compare
v
Contraseña válida
|
| jwt.sign
v
Token JWT
|
| Authorization: Bearer <token>
v
Endpoints protegidos
```

bcrypt se usa para no guardar contraseñas en texto plano. Cada contraseña se guarda como un hash con salt.

Ejecución local

```
cd api
npm install
node server.js
```

La API queda disponible en:

```
http://localhost:3000
```

Verificación rápida

```
curl http://localhost:3000/
```

Registro:

```
curl -X POST http://localhost:3000/auth/register \
-H "Content-Type: application/json" \
-d '{ "email": "admin@test.com", "password": "123456" }'
```

Login:

```
curl -X POST http://localhost:3000/auth/login \
-H "Content-Type: application/json" \
```

```
-d '{ "email": "admin@test.com", "password": "123456" }'
```

Consultar dominios con token:

```
curl http://localhost:3000/domains \
-H "Authorization: Bearer <token>"
```

UI React + Vite + Tailwind CSS

Descripción

La UI es la aplicación web administrativa del sistema. Permite que los administradores configuren dominios, URLs, políticas de caché, TTL, API Keys, usuarios y mecanismos de autenticación.

Fue implementada con React, Vite y Tailwind CSS. También se utilizó inteligencia artificial generativa como apoyo durante el diseño e implementación.

Responsabilidades principales

La UI se encarga de:

- Registrar administradores.
- Iniciar sesión.
- Cerrar sesión.
- Proteger rutas internas.
- Mostrar dashboard administrativo.
- Administrar dominios.
- Verificar dominios con TXT.
- Administrar URLs.
- Configurar tamaño de caché por URL.
- Configurar TTL por tipo de archivo.
- Seleccionar política de reemplazo.
- Administrar API Keys.
- Administrar usuarios por URL.
- Mostrar formulario público de autenticación para Zonal Cache.

Tecnologías usadas

Tecnología	Propósito
React	Construcción de componentes.
Vite	Servidor de desarrollo y empaquetado.
Tailwind CSS	Estilos.

Tecnología	Propósito
React Router	Navegación.
Axios	Cliente HTTP.
Zustand	Manejo de estado global.
Vercel	Despliegue automático.

Rutas principales

```
/login
/register
/dashboard
/dashboard/domains
/dashboard/domains/:domainId/urls
/dashboard/domains/:domainId/urls/:urlId/credentials
/auth
```

Flujo del login administrativo

```
Administrador
  |
  | /login
  v
UI
  |
  | POST /auth/login
  v
REST API Node.js
  |
  | Valida con Firebase
  v
Token JWT
  |
  | localStorage
  v
Dashboard
```

Formulario de autenticación para Zonal Cache

La ruta `/auth` es pública y se usa cuando un recurso protegido requiere usuario y contraseña.

Ejemplo:

```
http://localhost:5173/auth?domain=localhost&redirect=http://localhost:8080
```

Flujo:

```
Zonal Cache
|
| Redirige a /auth
v
Usuario ingresa username/password
|
| POST /auth/verify
v
REST API Node.js
|
| Valida credenciales
v
Devuelve token
|
| Redirección o muestra token
v
Zonal Cache valida sesión
```

Sistema de mocks

La UI incluye un sistema de mocks para desarrollo sin backend real.

Variable:

```
VITE_USE MOCK=true
```

Cuando el valor es `false`, la UI usa la API real configurada en:

```
VITE_API_URL=https://2026-01-2022437963-ic-7602.vercel.app
```

Ejecución local

```
cd ui
npm install
npm run dev
```

Abrir:

```
http://localhost:5173
```

REST API Java

Descripción

La REST API Java fue implementada como servidor origen de prueba para la Zonal Cache. Su objetivo es tener un backend real que responda datos JSON y permita probar los verbos HTTP principales: GET, POST, PUT y DELETE.

Esta API no se conecta con Firebase porque no administra la configuración del sistema. Su función es servir como origen de datos para demostrar que la Zonal Cache puede consultar un backend, guardar la respuesta y servirla desde caché.

Tecnologías usadas

Tecnología	Propósito
Java 17	Lenguaje de programación.
Spring Boot	Framework para crear la API REST.
Maven	Gestión de dependencias y build.
JUnit	Pruebas unitarias.
MockMvc	Pruebas de endpoints REST.
Docker	Contenerización.
Helm	Despliegue en Kubernetes.

Estructura

```
java-rest-api/  
├── Dockerfile  
├── pom.xml  
├── README.md  
└── src/  
    ├── main/  
    │   ├── java/redes/api/  
    │   │   ├── JavaRestApiApplication.java  
    │   │   └── controller/  
    │   │       ├── HealthController.java  
    │   │       └── ProductController.java  
    │   │   ├── dto/  
    │   │       └── ProductRequest.java  
    │   │   ├── model/  
    │   │       └── Product.java  
    │   │   └── service/  
    │   │       └── ProductService.java  
    │   └── resources/  
    │       └── application.properties  
    └── test/
```



```
└─ java/redes/api/  
    └─ JavaRestApiApplicationTests.java  
    └─ controller/  
        └─ ProductControllerTests.java  
    └─ service/  
        └─ ProductServiceTests.java
```

Endpoints implementados

Método	Ruta	Descripción
GET	/api/health	Verifica que la API esté activa.
GET	/api/products	Lista todos los productos.
GET	/api/products/{id}	Obtiene un producto por ID.
POST	/api/products	Crea un producto.
PUT	/api/products/{id}	Actualiza un producto.
DELETE	/api/products/{id}	Elimina un producto.

Almacenamiento de datos

Los datos se almacenan en memoria usando un `ConcurrentHashMap`.

Esto significa que:

- no se usa una base de datos externa;
- los datos se reinician cuando se reinicia la aplicación;
- es suficiente para este proyecto porque funciona como servidor origen de prueba;
- permite demostrar respuestas JSON y operaciones CRUD.

Productos iniciales:

```
1 -> Router  
2 -> Switch  
3 -> Access Point
```

Ejecución local con Docker

Desde la raíz del proyecto:

```
docker build -t java-rest-api:latest ./java-rest-api  
docker run --rm -p 8080:8080 java-rest-api:latest
```

Probar:

```
curl http://localhost:8080/api/health
curl http://localhost:8080/api/products
```

Ejecución con Kubernetes y Helm

Instalar:

```
helm upgrade --install java-rest-api ./charts/java-rest-api
```

Verificar:

```
kubectl get pods
kubectl get svc
```

Exponer localmente:

```
kubectl port-forward svc/java-rest-api-service 8081:8080
```

Probar:

```
curl http://localhost:8081/api/health
curl http://localhost:8081/api/products
```

Comandos para probar endpoints

Crear producto:

```
curl -X POST http://localhost:8081/api/products \
-H "Content-Type: application/json" \
-d '{
  "name": "Firewall",
  "description": "Firewall de prueba creado desde curl",
  "price": 55000,
  "stock": 3
}'
```

Actualizar producto:

```
curl -X PUT http://localhost:8081/api/products/1 \  
-H "Content-Type: application/json" \  
-d '{  
  "name": "Router actualizado",  
  "description": "Router modificado con PUT",  
  "price": 39000,  
  "stock": 8  
'
```

Eliminar producto:

```
curl -X DELETE http://localhost:8081/api/products/2
```

Pruebas unitarias

Como Maven no siempre está instalado localmente, las pruebas se pueden correr usando Docker:

```
cd java-rest-api  
docker run --rm -v "${PWD}:/app" -w /app maven:3.9.9-eclipse-temurin-17 mvn test
```

Resultado esperado:

```
Tests run: 17, Failures: 0, Errors: 0, Skipped: 0  
BUILD SUCCESS
```

URL interna para Kubernetes

Dentro del cluster, otros componentes pueden consultar la API usando:

```
http://java-rest-api-service:8080/api/products
```

Apache Server

Descripción

El Apache Server funciona como servidor origen de prueba para contenido estático. Sirve una página HTML básica con CSS y animaciones visuales relacionadas con redes.

Su función dentro del proyecto es permitir que la Zonal Cache pruebe almacenamiento de recursos estáticos, como archivos HTML y CSS.

Tecnologías usadas

Tecnología	Propósito
Apache HTTP Server	Servidor web.
HTML	Página estática.
CSS	Estilos y animaciones.
Docker	Contenerización.
Helm	Despliegue en Kubernetes.

Estructura

```
apache-server/  
├── Dockerfile  
├── index.html  
├── style.css  
└── README.md
```

Dockerfile

```
FROM httpd:2.4  
  
COPY index.html /usr/local/apache2/htdocs/index.html  
COPY style.css /usr/local/apache2/htdocs/style.css  
  
EXPOSE 80
```

Ejecución local con Docker

Desde la raíz del proyecto:

```
docker build -t apache-server:latest ./apache-server  
docker run --rm -p 8082:80 apache-server:latest
```

Abrir en el navegador:

```
http://localhost:8082
```

También se puede probar con curl:

```
curl http://localhost:8082
curl http://localhost:8082/style.css
```

Ejecución con Kubernetes y Helm

Instalar:

```
helm upgrade --install apache-server ./charts/apache-server
```

Verificar:

```
kubectl get pods
kubectl get svc
```

Exponer localmente:

```
kubectl port-forward svc/apache-server-service 8084:80
```

Probar:

```
curl http://localhost:8084
curl http://localhost:8084/style.css
```

URL interna para Kubernetes

Dentro del cluster, otros componentes pueden consultar Apache usando:

```
http://apache-server-service/
http://apache-server-service/style.css
```

Firestore

Descripción

Firebase Firestore se usa como base de datos principal para almacenar la configuración del sistema. La UI y la API Node.js permiten modificar esta configuración, mientras que la Zonal Cache la consulta para saber cómo debe comportarse.

Firebase almacena información como:

- usuarios administradores;
- dominios registrados;
- estado de verificación TXT;
- URLs configuradas;
- TTL;
- tamaño de caché;
- política de reemplazo;
- tipo de autenticación;
- API Keys;
- usuarios y contraseñas asociados a URLs.

Estructura general

```
users/  
└─ {userId}  
    ├── email  
    ├── password  
    ├── role  
    └─ createdAt  
  
domains/  
└─ {domain}  
    ├── domain  
    ├── ttl  
    ├── cache_size_mb  
    ├── replacement_policy  
    ├── auth_type  
    ├── api_key  
    ├── verified  
    ├── txtRecord  
    └─ urls/  
        └─ {urlId}  
            ├── pattern  
            ├── cacheSize  
            ├── fileTypes  
            ├── authType  
            ├── createdAt  
            ├── apikeys/  
            └─ users/
```

Campos importantes

Campo	Descripción
<code>ttl</code>	Tiempo de vida del recurso.
<code>cache_size_mb</code>	Tamaño máximo del caché.
<code>replacement_policy</code>	Política de reemplazo: LRU, LFU, FIFO, MRU o Random.
<code>auth_type</code>	Tipo de autenticación: none, api_key o session.
<code>api_key</code>	Clave para validar peticiones por header.
<code>txtRecord</code>	Registro usado para verificar propiedad del dominio.

Ejecución del proyecto

Requisitos previos

Para ejecutar el proyecto se necesita:

- Docker Desktop.
- Kubernetes habilitado.
- Helm instalado.
- kubectl instalado.
- Node.js 18 o superior.
- npm 9 o superior.
- Java 17 si se desea ejecutar la API Java sin Docker.
- Rust y Cargo si se desea ejecutar los servicios Rust sin Docker.
- Proyecto Firebase con Firestore habilitado.
- Variables de entorno configuradas.

Ejecución local por componentes

API Node.js

```
cd api
npm install
node server.js
```

Disponible en:

```
http://localhost:3000
```

UI

```
cd ui
npm install
npm run dev
```

Disponible en:

```
http://localhost:5173
```

Zonal Cache

```
cd zonal-cache
cargo run
```

Disponible en:

```
http://localhost:8080
```

REST API Java

```
docker build -t java-rest-api:latest ./java-rest-api
docker run --rm -p 8080:8080 java-rest-api:latest
```

Disponible en:

```
http://localhost:8080
```

Apache Server

```
docker build -t apache-server:latest ./apache-server
docker run --rm -p 8082:80 apache-server:latest
```

Disponible en:

```
http://localhost:8082
```


Despliegue con Docker y Helm

Construir imágenes

```
docker build -t dns-interceptor:latest ./dns-interceptor
docker build -t zonal-cache:latest ./zonal-cache
docker build -t java-rest-api:latest ./java-rest-api
docker build -t apache-server:latest ./apache-server
docker build -t ui:latest ./ui
```

Instalar servicios con Helm

```
helm upgrade --install dns-interceptor ./charts/dns-interceptor
helm upgrade --install zonal-cache ./charts/zonal-cache
helm upgrade --install java-rest-api ./charts/java-rest-api
helm upgrade --install apache-server ./charts/apache-server
helm upgrade --install ui ./charts/ui
```

Verificar recursos en Kubernetes

```
kubectl get pods
kubectl get svc
```

Probar servicios con port-forward

API Java:

```
kubectl port-forward svc/java-rest-api-service 8081:8080
curl http://localhost:8081/api/health
curl http://localhost:8081/api/products
```

Apache:

```
kubectl port-forward svc/apache-server-service 8084:80
curl http://localhost:8084
curl http://localhost:8084/style.css
```

Pruebas realizadas

Prueba 1 — Verificar REST API Node.js

Comando:

```
curl http://localhost:3000/
```

Resultado esperado:

```
{  
  "message": "API running"  
}
```

Conclusión:

La API Node.js responde correctamente y queda lista para recibir peticiones de la UI y la Zonal Cache.

Prueba 2 — Registro de usuario administrador

Pasos:

1. Abrir la UI en `/register`.
2. Ingresar correo y contraseña.
3. Confirmar el registro.
4. Verificar que se redirige al dashboard.

Resultado esperado:

El usuario administrador se crea correctamente y la contraseña queda almacenada con hash.

Conclusión:

El registro administrativo funciona y permite crear usuarios para administrar el sistema.

Prueba 3 — Login y logout administrativo

Pasos:

1. Abrir `/login`.
2. Ingresar credenciales válidas.
3. Verificar acceso al dashboard.
4. Recargar la página.
5. Cerrar sesión.
6. Intentar acceder nuevamente a `/dashboard`.

Resultado esperado:

La sesión se mantiene al recargar y se elimina correctamente al cerrar sesión.

Conclusión:

El flujo de autenticación administrativa funciona correctamente.

Prueba 4 — Protección de endpoints con JWT

Sin token:

```
curl http://localhost:3000/domains
```

Resultado esperado:

401 Token requerido

Con token:

```
curl http://localhost:3000/domains \  
-H "Authorization: Bearer <token>"
```

Resultado esperado:

Lista de dominios almacenados en Firebase.

Conclusión:

Los endpoints protegidos validan correctamente el token JWT.

Prueba 5 — Crear dominio con verificación TXT

Pasos:

1. Entrar al dashboard.
2. Ir a Dominios.
3. Crear un dominio.
4. Verificar que se genera un registro TXT.
5. Intentar la verificación.

Resultado esperado:

El dominio se crea en Firebase y se genera un TXT para validar propiedad.

Conclusión:

El sistema permite registrar dominios y preparar la verificación por TXT.

Prueba 6 — CRUD de URLs

Pasos:

1. Entrar a un dominio.
2. Crear una URL con wildcard.
3. Configurar tamaño de caché.
4. Configurar TTL.
5. Seleccionar política de reemplazo.
6. Editar la URL.
7. Eliminar la URL.

Resultado esperado:

La URL se crea, modifica y elimina correctamente.

Conclusión:

La UI y la API permiten administrar URLs y sus políticas de caché.

Prueba 7 — Gestión de API Keys

Pasos:

1. Crear una URL con autenticación por API Key.
2. Ir a Credenciales.
3. Generar una nueva API Key.
4. Eliminar la API Key.

Resultado esperado:

La clave se genera con formato esperado y luego se elimina correctamente.

Conclusión:

El sistema permite administrar credenciales de tipo API Key.

Prueba 8 — Gestión de usuarios por URL

Pasos:

1. Crear una URL con autenticación por usuario y contraseña.
2. Crear un usuario.
3. Editar el usuario.
4. Eliminar el usuario.

Resultado esperado:

El usuario se crea, actualiza y elimina correctamente.

Conclusión:

El sistema permite administrar usuarios asociados a URLs protegidas.

Prueba 9 — Autenticación por API Key en Zonal Cache

Configurar en Firebase:

auth_type=api_key

```
api_key=abc123
```

Probar sin API Key:

```
Invoke-WebRequest http://localhost:8080 -UseBasicParsing
```

Resultado esperado:

```
401 Unauthorized
```

Probar con API Key:

```
Invoke-WebRequest `
  -Uri http://localhost:8080 `
  -Headers @{"x-api-key"="abc123"} `
  -UseBasicParsing
```

Resultado esperado:

```
200 OK
```

Conclusión:

```
La Zonal Cache valida correctamente el encabezado x-api-key.
```

Prueba 10 — Autenticación por sesión en Zonal Cache

Configurar en Firebase:

```
auth_type=session
```

Abrir:

```
http://localhost:8080
```

Resultado esperado:

La Zonal Cache redirige al formulario de autenticación de la UI.

Credenciales de prueba:

```
usuario1  
123456
```

Conclusión:

El flujo de sesión permite validar usuarios desde la UI y luego autorizar el acceso en la Zonal Cache.

Prueba 11 — HIT/MISS en Zonal Cache

Comando de ejemplo:

```
curl "http://localhost:3000/cache?url=http://example.com"
```

Primera solicitud:

MISS: el recurso no existe en caché, se consulta el origen y se guarda.

Segunda solicitud:

HIT: el recurso ya existe en caché y se sirve desde disco.

Conclusión:

El motor de caché permite distinguir entre recursos nuevos y recursos previamente almacenados.

Prueba 12 — Algoritmos de reemplazo

Comando:

```
cd zonal-cache  
cargo test
```

Resultado esperado:

```
fifo_borra_el_primero_en_llegar  
fifo_ignora_accesos_recientes  
lru_borra_el_menos_reciente  
mru_borra_el_mas_reciente  
lfu_borra_el_menos_frecuente  
random_borra_alguna_entrada
```

Conclusión:

Las pruebas unitarias validan que las políticas FIFO, LRU, MRU, LFU y Random se comportan correctamente.

Prueba 13 — REST API Java con Docker

Comandos:

```
docker build -t java-rest-api:latest ./java-rest-api  
docker run --rm -p 8080:8080 java-rest-api:latest
```

Pruebas:

```
curl http://localhost:8080/api/health  
curl http://localhost:8080/api/products
```

Resultado esperado:

/api/health responde status UP.
/api/products devuelve productos en JSON.

Conclusión:

La REST API Java funciona como servidor origen de prueba para respuestas JSON.

Prueba 14 — REST API Java con Helm

Comandos:

```
helm upgrade --install java-rest-api ./charts/java-rest-api
kubectl get pods
kubectl get svc
kubectl port-forward svc/java-rest-api-service 8081:8080
```

Pruebas:

```
curl http://localhost:8081/api/health
curl http://localhost:8081/api/products
```

Resultado esperado:

La API responde correctamente desde Kubernetes.

Conclusión:

La REST API Java queda desplegada correctamente mediante Helm.

Prueba 15 — Pruebas unitarias de Java

Comando:

```
cd java-rest-api
docker run --rm -v "${PWD}:/app" -w /app maven:3.9.9-eclipse-temurin-17 mvn test
```

Resultado observado:

```
Tests run: 17, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

Conclusión:

Las pruebas unitarias de la API Java validan el servicio, los endpoints y el manejo de errores.

Prueba 16 — Apache Server con Docker

Comandos:

```
docker build -t apache-server:latest ./apache-server
docker run --rm -p 8082:80 apache-server:latest
```

Abrir:

```
http://localhost:8082
```

También:

```
curl http://localhost:8082
curl http://localhost:8082/style.css
```

Resultado esperado:

Apache devuelve el HTML y el CSS de la página.

Conclusión:

Apache funciona como servidor origen de contenido estático.

Prueba 17 — Apache Server con Helm

Comandos:

```
helm upgrade --install apache-server ./charts/apache-server
kubectl get pods
kubectl get svc
kubectl port-forward svc/apache-server-service 8084:80
```

Pruebas:

```
curl http://localhost:8084  
curl http://localhost:8084/style.css
```

Resultado esperado:

El servicio Apache responde correctamente desde Kubernetes.

Conclusión:

El Apache Server queda desplegado correctamente mediante Helm.

Recomendaciones

1. Mantener todos los valores configurables mediante variables de entorno, especialmente URLs, puertos, JWT_SECRET y credenciales de Firebase.
2. No subir credenciales reales al repositorio. Usar `.env.example` para mostrar la estructura esperada.
3. Ejecutar pruebas unitarias antes de construir imágenes Docker finales.
4. Probar primero cada componente de forma aislada antes de hacer pruebas de integración.
5. Mantener actualizados los Helm Charts, ya que el proyecto depende de la automatización con Kubernetes.
6. Usar nombres claros para los Services de Kubernetes, como `java-rest-api-service` y `apache-server-service`.
7. Documentar todos los endpoints con método, ruta, body esperado y respuesta esperada.
8. Probar la Zonal Cache con contenido JSON, HTML, CSS, HTTP externo y HTTPS externo.
9. Mantener logs descriptivos en cada componente para facilitar la defensa del proyecto.
10. Coordinar temprano los contratos entre DNS Interceptor, Zonal Cache y API Node.js.

Conclusiones

1. El proyecto permitió aplicar conceptos de capa de aplicación y capa de transporte usando HTTP, DNS, TCP y UDP.

2. La arquitectura por componentes ayudó a separar responsabilidades entre DNS Interceptor, Zonal Cache, UI, APIs y servidores origen.
 3. La Zonal Cache es el componente central del sistema porque integra caché en disco, autenticación, configuración dinámica y comunicación con servidores origen.
 4. Los algoritmos de reemplazo permiten administrar el espacio limitado del caché y decidir qué recurso eliminar cuando se alcanza el límite configurado.
 5. Firebase Firestore facilitó la administración dinámica de dominios, URLs, políticas de caché y credenciales.
 6. La UI permite administrar el sistema de forma visual, evitando modificar manualmente la base de datos.
 7. JWT es útil para proteger endpoints administrativos porque permite manejar sesiones sin guardar estado en el servidor.
 8. bcrypt mejora la seguridad al evitar que las contraseñas se almacenen en texto plano.
 9. La REST API Java permitió probar respuestas dinámicas en formato JSON mediante endpoints GET, POST, PUT y DELETE.
 10. El Apache Server permitió probar contenido estático como HTML y CSS.
-